

Beyond Classification: Evaluating LLMs for Structured Usability Issue Extraction from Mobile App Bug Reports

Turki Albalawi
University of North Texas
turkialbalawi@my.unt.edu

Renee Bryce
University of North Texas
renee.bryce@unt.edu

Abstract—Understanding how software bugs affect user experience and how well large language models (LLMs) can detect them is critical for building usable, high-quality mobile applications. In this study, we analyze 370 user reviews to assess the impact of bugs on usability and evaluate the effectiveness of state-of-the-art LLMs—including GPT-4 Turbo, Claude 3 Opus, Gemini 1.5 Pro, and LLaMA 3—in extracting usability-related information. Our results show that the most common impacts are unexpected behavior and blocked access to core features. Among the evaluated models, GPT-4 consistently achieves the highest accuracy across bug type classification, UI component extraction, and heuristic violation detection. Our findings highlight the promise of instruction-tuned LLMs in automating usability analysis and improving mobile app quality through user feedback. A replication package is available at <https://doi.org/10.5281/zenodo.15420652>.

Index Terms—Large Language Models (LLMs), Usability Bug Reports, Mobile Application Testing

I. INTRODUCTION

User reviews serve as a rich yet often underexploited source of feedback on how mobile applications behave in real-world use. Every day, millions of users leave reviews on platforms like the Google Play Store and Apple’s App Store, describing their experiences, frustrations, and expectations. These reviews capture not only technical problems such as bugs or crashes but also usability issues that directly affect the quality of interaction. Unlike structured user-reported bug submitted through formal issue tracking systems, user reviews tend to be informal, subjective, and context-rich, making them a unique lens for understanding how end-users actually perceive and interact with mobile applications [1], [2].

Prior work has leveraged app reviews to detect a variety of issues—such as crashes, performance failures, accessibility complaints, and feature requests. However, one area that continues to receive limited attention is usability—a critical quality attribute in mobile applications, where users expect interfaces to be fast, intuitive, and responsive. Usability violations, such as unclear feedback, poor layout, or inaccessible interface components, can result in user dissatisfaction, app abandonment, and negative ratings [3], [4]. Despite its importance, usability is often underrepresented in bug tracking systems and even in automated issue classification pipelines. Although user

studies offer rich insights, they are costly and time-consuming. In contrast, mining user-generated bug reports offers a scalable and low-cost way to augment usability testing. Studies have shown that even when reviews clearly describe an experience-related breakdown, they rarely frame it in usability terms (e.g., “visibility of system status” or “user control and freedom”) [5], [6]. Moreover, these reviews often omit precise technical indicators, such as which UI component was affected or what triggered the issue.

To address these limitations, recent work has begun to explore the use of large language models (LLMs) for usability classification. For example, Alsanousi et al. [7] investigated a range of fine-tuned and base LLMs for detecting heuristic violations in user reviews. Their work focused on binary classification of whether a usability heuristic (e.g., Nielsen’s principles) was violated. Similarly, Xiang et al. [8] introduced SimUser, an LLM-based tool that simulates diverse users interacting with mobile applications to generate heuristic usability feedback. Their method used a dual-agent system to emulate different user characteristics and evaluate interface designs through simulated user flows. These studies demonstrate that LLMs, when fine-tuned or guided by well-structured prompts, can be powerful tools for identifying usability issues, particularly when traditional rule-based approaches fall short.

While these contributions represent significant progress, they focus primarily on high-level usability feedback—typically identifying whether a heuristic is violated. However, our observation of real-world user reviews shows that these reviews often embed more detailed and actionable information. For example, consider the review: 🗨️ “Ad bugs make it unusable; after watching the ad, it goes back to the prompt menu and won’t show results after clicking the button.” This review contains multiple layers of insight: it implies a bug type (a faulty ad-related interaction), identifies the affected UI component (a button), and reflects a clear usability breakdown (lack of system feedback or loss of user control). Capturing this rich, structured information can offer developers more actionable guidance than simply flagging the review as usability-related.

Recent work in prompt-based learning and LLM-powered code and text understanding supports this possibility. For instance, Pister et al. [9] showed how GPT-4 can accurately

categorize developer-written prompts in software development contexts. Tafreshipour et al. [10] demonstrated that LLMs can extract task-specific details with high precision from unstructured text, reinforcing their utility in context-sensitive information retrieval.

Building on this foundation, we propose a new approach that goes beyond detecting usability violations. Unlike prior work, we extract structured information—bug type, affected UI component, and violated heuristic—providing insight for developers. We investigate how LLMs can be used to extract structured, multi-dimensional usability information from user reviews, specifically: i) The type or symptom of the bug, ii) the UI component affected, and iii) the usability heuristic violated.

By capturing this triad of information, we aim to bridge the gap between general user frustration and actionable insights that developers and designers can use to improve mobile applications. Unlike prior work [7] that treats usability as a binary classification task, our approach focuses on structured extraction to better support root cause analysis, design prioritization, and automated triaging of user feedback.

To investigate the feasibility and accuracy of this approach, our study is guided by the following research questions:

RQ1: To what extent do user-reported bugs impact the perceived usability of mobile applications?

RQ2: How accurately can LLMs identify usability-related bugs, extract the affected UI components, and map issues usability heuristics?

The remainder of the paper is organized as follows: Section II reviews related work. Section III describes our methodology, and Section IV presents the results. Followed by study limitations in Section V. Finally, Section VI concludes the paper and outlines future work.

II. RELATED WORK

Recent work has increasingly applied LLMs to mobile app review analysis, particularly for classification tasks. For example, Palmeshofer et al. [11] used Mistral models with zero-shot, one-shot, and few-shot prompting to classify app reviews, finding that few-shot prompting improved results—especially for smaller models. However, their approach does not identify usability bugs or categorize them by bug type, affected UI element, or violated heuristic. In contrast, our method performs structured, multi-field classification that supports more actionable usability insights.

Some studies have explored usability-specific applications. Pourasad et al. introduced UX-LLM to predict usability problems in iOS apps [12], but did not map them to specific heuristics or affected UI components [13]. Our approach goes beyond prediction by mapping usability problems to Nielsen’s heuristics and extracting structured fields—App Component, UI Element Type, Bug Type, and Heuristic Violated—providing detailed insight for developer to enhance usability. This improves the value of usability.

Earlier work laid the foundation for automated review analysis using traditional machine learning and topic modeling

techniques [2], [13]–[15]. These methods were later extended by supervised learning approaches that improved classification precision and recall [16], and later by studies incorporating social media data [17] and active learning [18]. The field has since evolved with deep learning approaches such as CNNs and BERT, which enhanced performance in multilingual classification and aspect-based sentiment analysis [19]–[21].

Other studies have leveraged LLMs for extracting features from app reviews. Motger et al. [22] framed feature extraction as a supervised token classification problem, improving precision and recall. However, their focus was on general feature identification rather than usability-specific bug detection.

Finally, efforts to detect usability issues and heuristic violations have often relied on manual techniques. Diniz et al. [23] manually analyzed feedback to identify heuristic violations, limiting scalability and consistency. Alsanousi et al. [7] used fine-tuned LLMs for binary classification of heuristic violations, while Xiang et al. [8] introduced SimUser, which generates usability feedback through simulated user interaction but lacks structured classification fields.

In contrast, our work introduces a practical and scalable method that combines instruction-based prompting and few-shot learning to extract structured usability information. By identifying specific bug types, UI components, and heuristic violations, our approach offers deeper insight into user experience breakdowns and supports more informed development decisions.

III. METHODOLOGY

A. Data Collection

The dataset used in this study comprises 13,127 user reviews, curated from prior work on app review mining [4], [16], [24]. These reviews primarily focus on usability-reported issues and bugs across a wide range of application domains. The dataset includes high-usage applications from both the Google Play Store and the Apple App Store, and expanded to incorporate reviews from LLM-based applications [24] such as ChatGPT and Copilot.

B. Data cleaning and Usability Filtering

Data cleaning: The raw dataset, while focused on user-reported issues, required cleaning. Following prior work [4], we removed non-informative entries, symbolic content, and reviews lacking meaningful language, resulting in a final set of 3,212 reviews.

Usability Filtering: To investigate bugs and their impact on user-perceived usability, we filtered the initial set of 3,212 reviews to identify usability-related content [25], [26]. We applied a text-matching approach using regular expressions (Regex), derived from keyword patterns reported in prior work [4]. The results of applying the string matching to **370** reviews that contain both bugs and usability reported issues as our final set.

TABLE I: Dimensions, Definitions, Labels (*excluding* “Other”), and Frequency.

Dimension	Definition	Labels	% of the total (370)
Bug Type	Technical failure mentioned in the review.	• UI Unresponsive (24.59%) • Crash (22.16%) • Freeze (21.89%) • Error Message (21.35%)	89.99%
UI Component	UI element affected by the issue.	• AI Component (27.30%) • Button (17.03%) • Voice Input (15.95%) • Screen/Layout (11.89%) • Popup/Dialog (11.89%)	76.22%
Usability Heuristics	Violated usability principle (Nielsen).	• H5 – Error prevention (55.95%) • H1 – Visibility of system status (46.22%) • H3 – User control and freedom (32.97%) • H2 – Match with real world (23.51%)	98.65%

C. Manual Annotation

In this study, we aim to label the dataset along three key dimensions (RQ2): (i) the type or description of the bug, (ii) the user interface (UI) component affected, and (iii) the usability heuristic violated. To achieve this, we employed a manual labeling process guided by close inspection of the reviews, following an approach similar to that of McIlroy et al. [27]. Rather than relying on a predefined taxonomy, we conducted an open coding process to identify recurring patterns in bug types, UI elements, and heuristic violations, allowing emergent categories to be grouped meaningfully. Table I presents the final set of labels, their definitions, sub-labels, and the distribution across the dataset.

While labeling the reviews, we also recorded the extent to which each reported bug impacted usability (RQ1). During the annotation process, we identified four recurring usability impact themes (see Table II). These impact categories provide a structured way to assess how different types of bugs affect the overall usability of mobile applications.

Manual validation: To mitigate potential bias in manual annotation, we adopted a multi-rater validation process, following the methodology used in prior studies [27], [28]. Two qualified investigators independently labeled each review, and inter-rater reliability was assessed using Cohen’s Kappa coefficient [29]. The resulting Kappa score was 0.91, indicating a high level of agreement. Any discrepancies were resolved through a joint discussion with a third investigator, and the final label was assigned only upon full consensus among all three annotators.

D. LLMs and Prompt Design

Models: We evaluated four state-of-the-art LLMs: GPT-4 (gpt-4-turbo), Gemini 1.0 Pro, Llama 3 (Llama-3.1-70B-Instruct), and Claude 3 Opus. These models represent some of the latest advancements in natural language and code understanding. Our selection is based on a recent survey of LLMs [30] and their performance scores across multiple benchmarks as reported by their

TABLE II: Usability Impact Definitions and Frequency

Impact Type	Definition	Freq. (%)
Unexpected Behavior	Incorrect or unpredictable response that contradicts expectations.	55.68
Blocked Feature	Bug prevents task completion or feature access.	47.03
Unclear Interaction	Poor UI communication or confusing logic.	12.43
Hard to Control	Laggy/unresponsive behavior needing repeated actions.	10.27

respective official sources as follows: GPT-4¹, Gemini 1.0 Pro², Claude 3 Opus³, and Llama 3⁴. Moreover, GPT-4 and LLaMA 3 have shown strong performance in tasks involving UI structure understanding in mobile environments [31], [32], making them particularly relevant to our usability-focused task.

We tuned critical hyperparameters such as `temperature` and `top_p` to improve output consistency and quality. Following guidance from Arora et al. [33], we used a low temperature (0.1) and a top-p of 0.75, settings that balance determinism and diversity and are shown to yield more accurate and reliable results in classification and extraction tasks.

Prompt Design: Given the nature of our task—which involves identifying bug types, extracting UI components (e.g., buttons or screens), and mapping issues to usability heuristics—we designed prompts that integrate task-specific instructions with illustrative examples. This design follows best practices from recent prompt engineering research [34], incorporating key elements such as role specification, explicit task constraints, and structured output formatting [35], [36]. We adopted an *instruction-based prompting approach enhanced with few-shot examples* [37].

¹<https://github.com/openai/simple-evals>

²<https://deepmind.google/technologies/gemini/pro/>

³<https://www.anthropic.com/news/claude-3-family>

⁴<https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>

Our prompt instructs the model to classify each review along four predefined dimensions—*bug type*, *UI component*, and *violated heuristic*—using only fixed label sets derived from the manually constructed labels (see Table II). The prompt explicitly listed the available labels for each dimension and instructed the model to output a valid json object using one or more applicable labels per field. If no label applied, the model was instructed to return "Other". For reproducibility, we provide the complete prompt template used in our experiments, including task instructions and example annotations⁵.

E. Post-Processing

To ensure fair and consistent evaluation, we systematically analyzed each model’s JSON output for structural correctness and completeness. Given that LLMs may occasionally return incomplete or empty responses due to issues like API timeouts [38], [39], we used Python’s json⁶ module to parse and validate key-value pairs. If a model failed to return a valid output, we applied a retry mechanism to resubmit the prompt and recover missing content.

F. Evaluation and Metrics

To evaluate LLM outputs, we performed exact set-based string matching between the model-generated labels and the manually annotated ground truth. For each classification field, we compared predicted and reference label sets using string equality, treating each label as an independent class. From these comparisons, we computed per-label **Precision**, **Recall**, and **F1 score** using standard multi-label evaluation procedures.

IV. RESULTS

The study results present a comprehensive evaluation of how well LLMs can identify and classify usability-related issues in mobile app reviews. We report findings across two core research questions: (1) the extent to which bugs impact user experience, and (2) the ability of LLMs to extract meaningful labels such as bug symptoms, affected UI components, and violated usability heuristics.

A. RQ1: To what extent do user-reported bugs impact the perceived usability of mobile applications?

To assess the extent to which bugs affect the user experience, we analyzed the distribution of *usability impact* in our dataset. Table III summarizes these findings.

The most frequent usability impact was **Unexpected or Incorrect Behavior** (55.68%). This category includes instances where the application generated responses or actions that conflicted with the user’s intent. These failures frequently originated from the *AI Component*, where underlying bugs disrupted the expected flow of interaction and affected key usability elements. For example, one user reported: “*The assistant kept switching the screens mid-conversation and replaced my response with unrelated text...*” In this case, integrating the AI assistant introduced instability, causing the interface to

change screens unexpectedly—likely due to improper context handling or unintended trigger conditions.

The second most common impact, **Blocked Access to Feature** (47.03%), was primarily caused by symptoms such as *Crash*, *Freeze*, and *UI Unresponsive*. These technical failures prevented users from completing core tasks. For instance, a user wrote: “*The app crashes every time I press the login button. I can’t even open the settings.*” This impact category reflects the most severe form of usability failure, where essential functionality is rendered inaccessible.

Unclear or Confusing Interaction (12.43%) was most often associated with bug symptoms like *UI Unresponsive*. These issues frequently involved vague or misleading UI behavior, such as invisible UI elements or hidden actions. One user noted: “*... I didn’t know you had to swipe left to delete – there’s no indicator or instruction.*” Such breakdowns typically stem from poor discoverability and violate principles of intuitive design.

Difficult to Control (10.27%) was largely driven by symptoms like *Slow Response* or repeated *UI Unresponsive* issues. These bugs affected the fluidity of user interaction, requiring users to perform actions multiple times or endure delays. An illustrative review stated: “*I have to tap the button three times before it works, and sometimes it lags for seconds.*”

Summary of RQ1: Bugs significantly impact user experience, most commonly through unexpected behavior and blocked access to core features. Other issues include confusing interactions and poor responsiveness, all of which undermine usability and user satisfaction.

B. RQ2: How accurately can LLMs identify usability-related bugs, extract the affected UI components, and map issues usability heuristics?

Table III presents a comparative evaluation of four selected LLMs on the task of classifying bug type/symptoms. The results reveal a consistent performance, with GPT-4 Turbo outperforming all other models across every bug category in terms of precision, recall, and F1 score. GPT-4 achieves the highest F1 scores in all labels, ranging from 89.1% (Error Message) to 93.4% (Crash), indicating strong generalization across bug types. Claude 3 follows closely behind, maintaining competitive precision and recall, but slightly lower than GPT-4 in all metrics. Its performance remains robust, particularly in the Error Message and Freeze categories, where its F1 scores (89.0% and 90.0%) are within 1–2 points of GPT-4. In contrast, Gemini 1.5 Pro and LLaMA 3 70B show a noticeable drop in performance. Gemini achieves mid-80s F1 scores, while LLaMA 3, the least accurate model, falls below 80% F1 in three of the four categories. Table IV presents the classification performance on identifying UI components mentioned in user reviews. Similar to bug classification performance, GPT-4 Turbo achieves the highest precision, recall, and F1 scores, consistently outperforming the other models across all categories. Its F1 scores range from 86.5% (Popup/Dialog) to 91.5% (AI Component). Claude 3 ranks second in

⁵<https://doi.org/10.5281/zenodo.15420652>

⁶<https://docs.python.org/3/library/json.html>

TABLE III: Performance comparison for **Bug Symptom** classification. P = Precision, R = Recall, F1 = F1 score (%).

Model	Crash			Freeze			Error Msg.			UI Unresp.		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
GPT-4 Turbo	94.2	92.6	93.4	91.5	90.8	91.1	89.9	88.3	89.1	90.7	91.2	90.9
Claude 3 Opus	92.7	91.4	92.0	90.4	89.7	90.0	90.1	87.9	89.0	88.8	89.1	89.0
Gemini 1.5 Pro	89.3	87.5	88.4	85.6	86.2	85.9	84.1	82.3	83.2	83.7	84.5	84.1
LLaMA 3 70B	86.4	84.2	85.3	81.7	80.4	81.0	78.9	79.6	79.2	79.5	78.3	78.9

TABLE IV: Performance comparison for **UI Component** classification. P = Precision, R = Recall, F1 = F1 score (%).

Model	AI Comp.			Button			Voice In.			Screen			Popup		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
GPT-4 Turbo	92.1	91.0	91.5	90.4	89.2	89.8	89.6	87.5	88.5	87.3	86.8	87.0	86.9	86.1	86.5
Claude 3 Opus	89.5	88.0	88.7	88.1	87.6	87.8	87.0	85.5	86.2	84.3	85.0	84.6	83.8	83.0	83.4
Gemini 1.5 Pro	86.2	85.0	85.6	84.9	84.2	84.5	82.1	83.4	82.7	80.4	81.0	80.7	79.5	78.8	79.1
LLaMA 3 70B	81.3	80.1	80.7	79.0	77.5	78.2	77.2	75.8	76.5	74.9	73.4	74.1	72.5	71.3	71.9

TABLE V: Performance comparison for **Heuristic classification**. P = Precision, R = Recall, F1 = F1 score (%).

Model	H5 – Error Prev.			H1 – Visibility			H3 – Control/Freedom			H2 – Real World		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
GPT-4 Turbo	91.4	90.6	91.0	89.7	88.3	89.0	88.5	87.0	87.7	87.1	86.0	86.5
Claude 3 Opus	90.0	88.9	89.4	87.6	86.8	87.2	86.1	85.3	85.7	84.5	83.1	83.8
Gemini 1.5 Pro	85.5	84.3	84.9	83.2	81.7	82.4	81.0	80.4	80.7	78.9	77.2	78.0
LLaMA 3 70B	80.3	79.5	79.9	78.0	76.2	77.1	75.6	74.1	74.8	72.3	71.0	71.6

performance, with stable F1 scores, and maintains competitive accuracy in identifying commonly referenced components like Button (87.8%) and Voice Input (86.2%). Gemini 1.5 Pro shows moderate performance, with F1 scores around 80–85%, while LLaMA 3 70B lags behind, particularly on visual layout and interaction components such as Screen/Layout (74.1%) and Popup/Dialog (71.9%). Table V compares the performance in classifying usability heuristic violations. Among the evaluated models, GPT-4 Turbo leads in all categories, achieving F1 scores ranging from 86.5% to 91.0%. Its highest performance is observed in H5 – Error Prevention (91.0%). These results also demonstrate that our prompt design for GPT-4, combined with carefully selected hyperparameter settings, outperforms prior work in usability heuristic identification [7]. Claude 3 Opus consistently ranks second, with strong F1 scores across all heuristics, particularly in H1 – Visibility of System Status (87.2%) and H3 – User Control and Freedom (85.7%). Its close performance to GPT-4 Turbo indicates its reliability in identifying common heuristic violations. Gemini 1.5 Pro shows moderate performance, maintaining F1 scores between 78.0% and 84.9%, while LLaMA 3 70B lags behind, with F1 scores falling below 80% in all categories.

Summary of RQ2: Instruction-tuned models, especially GPT-4 Turbo, showed the highest accuracy across all tasks, while open-source models like LLaMA 3 performed noticeably lower.

V. THREATS TO VALIDITY

We discuss the potential threats to the validity of our study using standard categories: construct, internal, external, and reliability validity. **Internal Validity.** Our LLM evaluation

depends on consistent JSON output parsing and retry mechanisms for handling incomplete responses. Although we used a standardized prompt design and controlled for API errors, response variability—especially in generative outputs—could affect consistency across runs. Moreover, the use of pre-defined few-shot examples may introduce prompt-specific bias that benefits certain models more than others. **External Validity.** This study focuses on a dataset of 370 mobile app reviews, including both general-purpose and LLM-based applications. While this provides diversity across domains, findings may not generalize to other apps, or user populations with different expectations. Furthermore, the evaluation covers only four LLMs; newer or domain-specific models may exhibit different behaviors, limiting generalizability beyond the selected model set.

VI. CONCLUSION AND FUTURE WORK

This study analyzes how bugs impact user experience in mobile apps and evaluates the effectiveness of modern LLMs in detecting these issues from user reviews. Our findings show that usability is often compromised by unexpected system behavior and feature-blocking failures. Instruction-tuned LLMs, particularly GPT-4 Turbo, outperform other models in extracting actionable usability insights. These results indicate that LLMs can help developers efficiently identify high-impact issues, leading to more resilient mobile software. Unlike previous approaches, our method extracts structured information—bug type, affected UI component, and violated heuristic—providing practical insights to enhance usability. In future work, we plan to explore alternative models, particularly open-source LLMs, as well as experiment with diverse

prompting techniques to further improve performance and generalization.

REFERENCES

- [1] M. Haering, C. Stanik, and W. Maalej, "Automatically matching bug reports with related app reviews," in *2021 IEEE/ACM 43rd international conference on software engineering (ICSE)*. IEEE, 2021, pp. 970–981.
- [2] D. Pagano and W. Maalej, "User feedback in the appstore: An empirical study," in *2013 21st IEEE international requirements engineering conference (RE)*. IEEE, 2013, pp. 125–134.
- [3] E. Bakiu and E. Guzman, "Which feature is unusable? detecting usability and user experience issues from user reviews," in *2017 IEEE 25th international requirements engineering conference workshops (REW)*. IEEE, 2017, pp. 182–187.
- [4] B. Alsanousi, A. S. Albeshier, H. Do, and S. Ludi, "Investigating the user experience and evaluating usability issues in ai-enabled learning mobile apps: An analysis of user reviews," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 6, 2023.
- [5] T. Hirsch and B. Hofer, "Identifying non-natural language artifacts in bug reports," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. IEEE, 2021, pp. 191–197.
- [6] A. Patil and A. Jadon, "Auto-labelling of bug report using natural language processing," in *2023 IEEE 8th international conference for convergence in technology (I2CT)*. IEEE, 2023, pp. 1–7.
- [7] B. Alsanousi, S. Ludi, and H. Do, "Lime-aided automated usability issue detection from user reviews: Leveraging llms for enhanced user experience analysis," in *ACHI 2024: The Seventeenth International Conference on Advances in Computer-Human Interactions*, 2024.
- [8] W. Xiang, H. Zhu, S. Lou, X. Chen, Z. Pan, Y. Jin, S. Chen, and L. Sun, "Simuser: Generating usability feedback by simulating various users interacting with mobile applications," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–17.
- [9] K. Pister, D. J. Paul, I. Joshi, and P. Brophy, "Promptset: A programmer's prompting dataset," in *Proceedings of the 1st International Workshop on Large Language Models for Code*, 2024, pp. 62–69.
- [10] M. Tafreshipour, A. Imani, E. Huang, E. Almeida, T. Zimmermann, and I. Ahmed, "Prompting in the wild: An empirical study of prompt evolution in software repositories," *arXiv preprint arXiv:2412.17298*, 2024.
- [11] M. Palmetshofer, D. C. Schedl, and A. Stöckl, "Optimizing app review classification with large language models: A comparative study of prompting techniques," in *2024 4th International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*. IEEE, 2024, pp. 1–6.
- [12] A. E. Poursad and W. Maalej, "Does genai make usability testing obsolete?" *arXiv preprint arXiv:2411.00634*, 2024.
- [13] M. V. Phong, T. T. Nguyen, H. V. Pham, and T. T. Nguyen, "Mining user opinions in mobile app reviews: A keyword-based approach (t)," in *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2015, pp. 749–759.
- [14] W. Maalej and H. Nabil, "Bug report, feature request, or simply praise? on automatically classifying app reviews," in *2015 IEEE 23rd international requirements engineering conference (RE)*. IEEE, 2015, pp. 116–125.
- [15] N. Chen, J. Lin, S. C. Hoi, X. Xiao, and B. Zhang, "Ar-miner: mining informative reviews for developers from mobile app marketplace," in *Proceedings of the 36th international conference on software engineering*, 2014, pp. 767–778.
- [16] W. Maalej, Z. Kurtanović, H. Nabil, and C. Stanik, "On the automatic classification of app reviews," *Requirements Engineering*, vol. 21, pp. 311–331, 2016.
- [17] E. Guzman, M. Ibrahim, and M. Glinz, "A little bird told me: Mining tweets for requirements and software evolution," in *2017 IEEE 25th international requirements engineering conference (RE)*. IEEE, 2017, pp. 11–20.
- [18] V. T. Dhinakaran, R. Pulle, N. Ajmeri, and P. K. Murukanniah, "App review analysis via active learning: reducing supervision effort without compromising classification accuracy," in *2018 IEEE 26th international requirements engineering conference (RE)*. IEEE, 2018, pp. 170–181.
- [19] C. Stanik, M. Haering, and W. Maalej, "Classifying multilingual user feedback using traditional machine learning and deep learning," in *2019 IEEE 27th international requirements engineering conference workshops (REW)*. IEEE, 2019, pp. 220–226.
- [20] N. Aslam, W. Y. Ramay, K. Xia, and N. Sarwar, "Convolutional neural network based classification of app reviews," *Ieee Access*, vol. 8, pp. 185 619–185 628, 2020.
- [21] S. Gunathilaka and N. De Silva, "Aspect-based sentiment analysis on mobile application reviews," in *2022 22nd International Conference on Advances in ICT for Emerging Regions (ICTer)*. IEEE, 2022, pp. 183–188.
- [22] Q. Motger, A. Miaschi, F. Dell'Orletta, X. Franch, and J. Marco, "Leveraging encoder-only large language models for mobile app review feature extraction," *Empirical Software Engineering*, vol. 30, no. 3, p. 104, 2025.
- [23] L. d. N. Diniz, J. C. de Souza Filho, and R. M. Carvalho, "Can user reviews indicate usability heuristic issues?" in *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, 2022, pp. 1–6.
- [24] R. Alabduljabbar, "User-centric ai: evaluating the usability of generative ai applications through user reviews on app stores," *PeerJ Computer Science*, vol. 10, p. e2421, 2024.
- [25] M. Booday and A. Albeshier, "Evaluating the usability of mobile applications: The case of covid-19 apps in saudi arabia," in *2021 22nd International Arab Conference on Information Technology (ACIT)*. IEEE, 2021, pp. 1–7.
- [26] P. Weichbroth and A. Baj-Rogowska, "Do online reviews reveal mobile application usability and user experience? the case of whatsapp," in *2019 Federated Conference on Computer Science and Information Systems (FedCSIS)*. IEEE, 2019, pp. 747–754.
- [27] S. McIlroy, N. Ali, H. Khalid, and A. E. Hassan, "Analyzing and automatically labelling the types of user issues that are raised in mobile app reviews," *Empirical Software Engineering*, vol. 21, pp. 1067–1106, 2016.
- [28] A. Aljohani and H. Do, "From fine-tuning to output: An empirical investigation of test smells in transformer-based test code generation," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, 2024, pp. 1282–1291.
- [29] M. L. McHugh, "Interrater reliability: the kappa statistic," *Biochemia medica*, vol. 22, no. 3, pp. 276–282, 2012.
- [30] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [31] D. Zhang, Z. Shen, R. Xie, S. Zhang, T. Xie, Z. Zhao, S. Chen, L. Chen, H. Xu, R. Cao *et al.*, "Mobileenv: Building qualified evaluation benchmarks for llm-gui interaction," 2024.
- [32] F. Mehralian, T. Barik, J. Nichols, and A. Swearngin, "Automated code fix suggestions for accessibility issues in mobile apps," 2024. [Online]. Available: <https://arxiv.org/abs/2408.03827>
- [33] C. Arora, A. I. Sayeed, S. Licorish, F. Wang, and C. Treude, "Optimizing large language model hyperparameters for code generation," *arXiv preprint arXiv:2408.10577*, 2024.
- [34] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *International conference on data intelligence and cognitive informatics*. Springer, 2023, pp. 387–402.
- [35] A. Deljouyi, R. Koohestani, M. Izadi, and A. Zaidman, "Leveraging large language models for enhancing the understandability of generated unit tests," *arXiv preprint arXiv:2408.11710*, 2024.
- [36] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. El-nashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023.
- [37] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [38] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, "An empirical study on challenges for openai developers," *arXiv preprint arXiv:2408.05002*, 2024.
- [39] X. Hou, Y. Zhao, and H. Wang, "Voices from the frontier: A comprehensive analysis of the openai developer forum," *arXiv preprint arXiv:2408.01687*, 2024.